

Computer Architecture: *Memory Basics*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- **How does the machine store data?**
- How does the machine interface using buses?
- What does it mean to have “allowed” memory?
- How does the machine speed up data access?

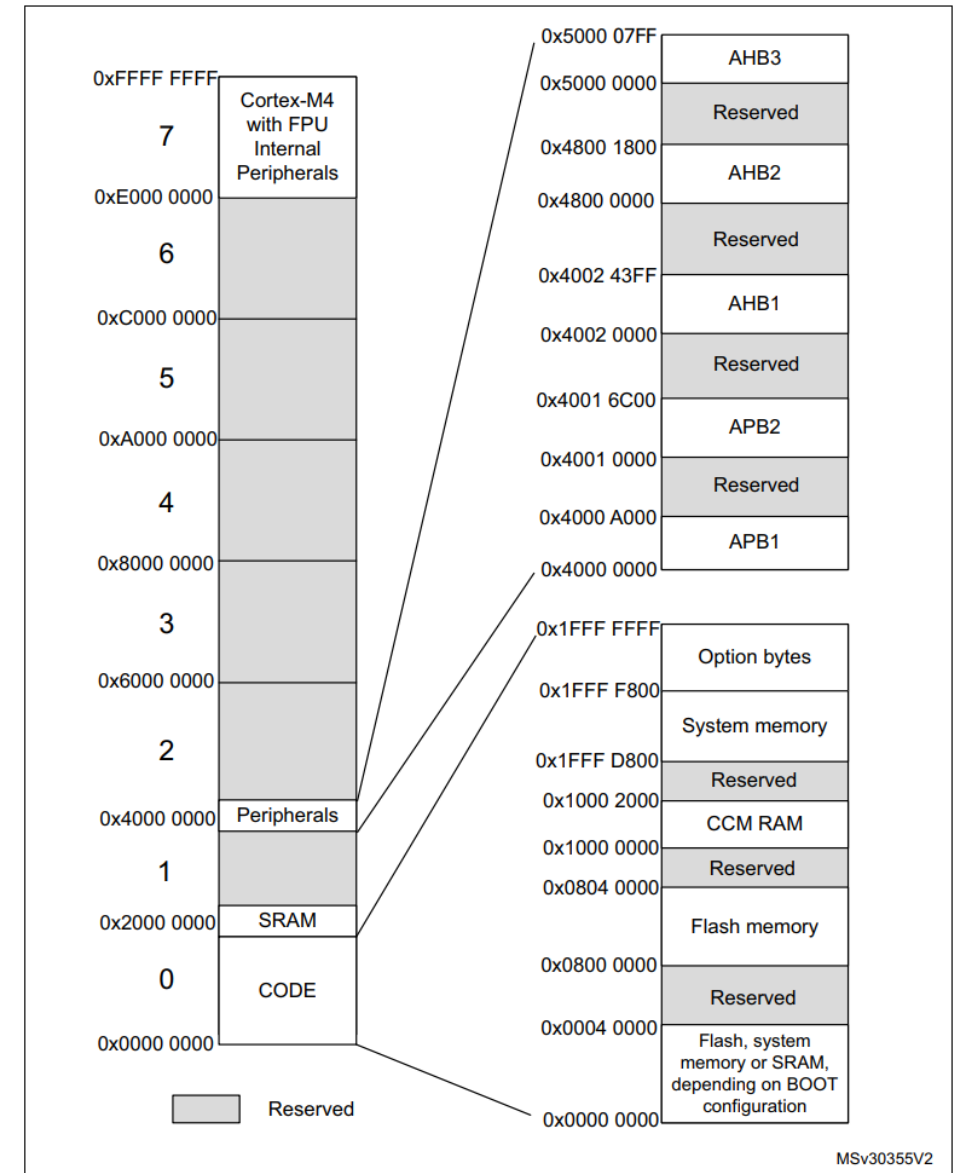
Memory

- What is memory?

Memory Addressing

- The entire memory space is defined by the number of bits in the address bus:
 - 16-bit = 65 KB (**65,536**)
 - 32-bit = 4 GB (**4,294,967,296**)
 - 64-bit = 16 EB (**18,446,744,073,709,551,616**)
- There is *no standard* when it comes to what address to use.
- Address “0” may be *invalid* – blocks get assigned anywhere in the address space. Sometimes for historical reasons.

Figure 8. STM32F303xB/STM32F303xC memory map



Memory Banking

- What happens if you run out of memory (e.g., 16-bit, 32-bit)?
 - Requires banking out memory
 - Older devices often have memory banks you switch between

Memory Width

- This is the width of the memory itself



Byte Addressing

Address: 0x20001000 Size: auto	
20001000	F7045DEE
20001004	B75C41C8
20001008	C215C947
2000100C	507E51E8
20001010	4BF8F388
20001014	8AD17A54
20001018	A2055766
2000101C	AA735032
20001020	5757652E
20001024	D2432A56
20001028	60530000

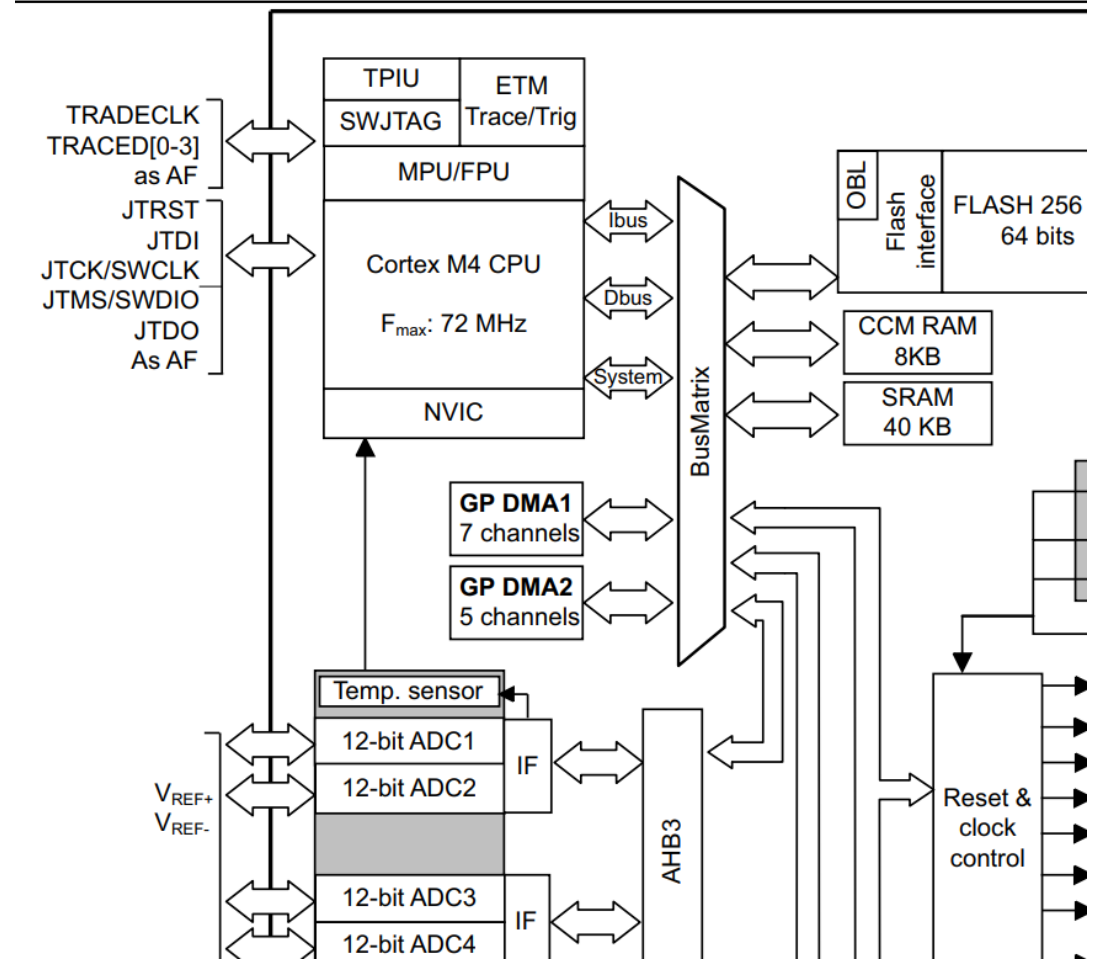
: i

Address: 0x20001000 Size: auto	
20001000	EE 5D 04 F7
20001004	C8 41 5C B7
20001008	47 C9 15 C2
2000100C	E8 51 7E 50
20001010	88 F3 F8 4B
20001014	54 7A D1 8A
20001018	66 57 05 A2
2000101C	32 50 73 AA
20001020	2E 65 57 57
20001024	56 2A 43 D2

Computer Data Width

- This is the width of the computer bus

Figure 1. STM32F303xB/STM32F303C8



Address Space Mapping

- Peripherals are mapped into address space – this may include many small RAMs for example or other small blocks.
- It may be possible to move these, often in datasheets/manuals you see registers as an offset for this reason:

21.4.3 TIMx slave mode control register (TIMx_SMCR)(x = 2 to 4)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SMS[3]
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			OCCS	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

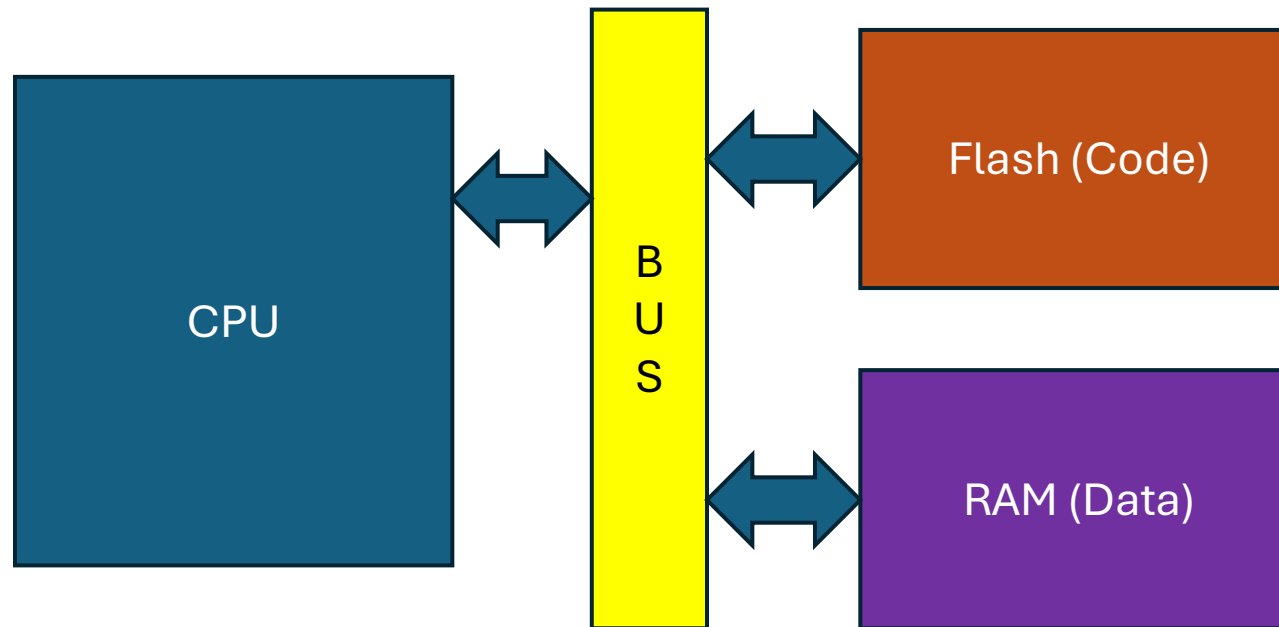
Bits 31:17 Reserved, must be kept at reset value.

Bit 15 **ETP**: External trigger polarity

This bit selects whether ETR or $\overline{\text{ETR}}$ is used for trigger operations

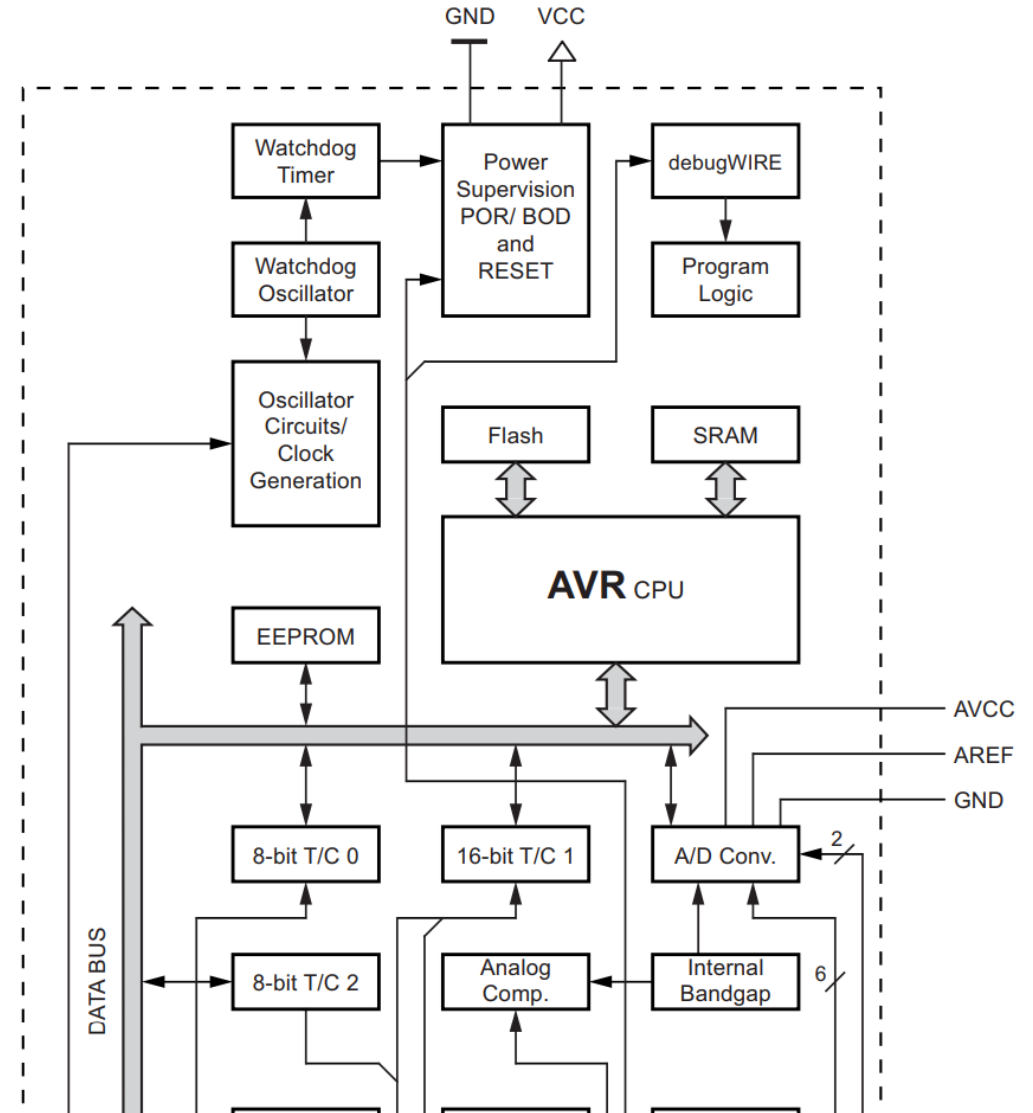
von Neumann Architecture

- One bus with everything



Harvard Architecture

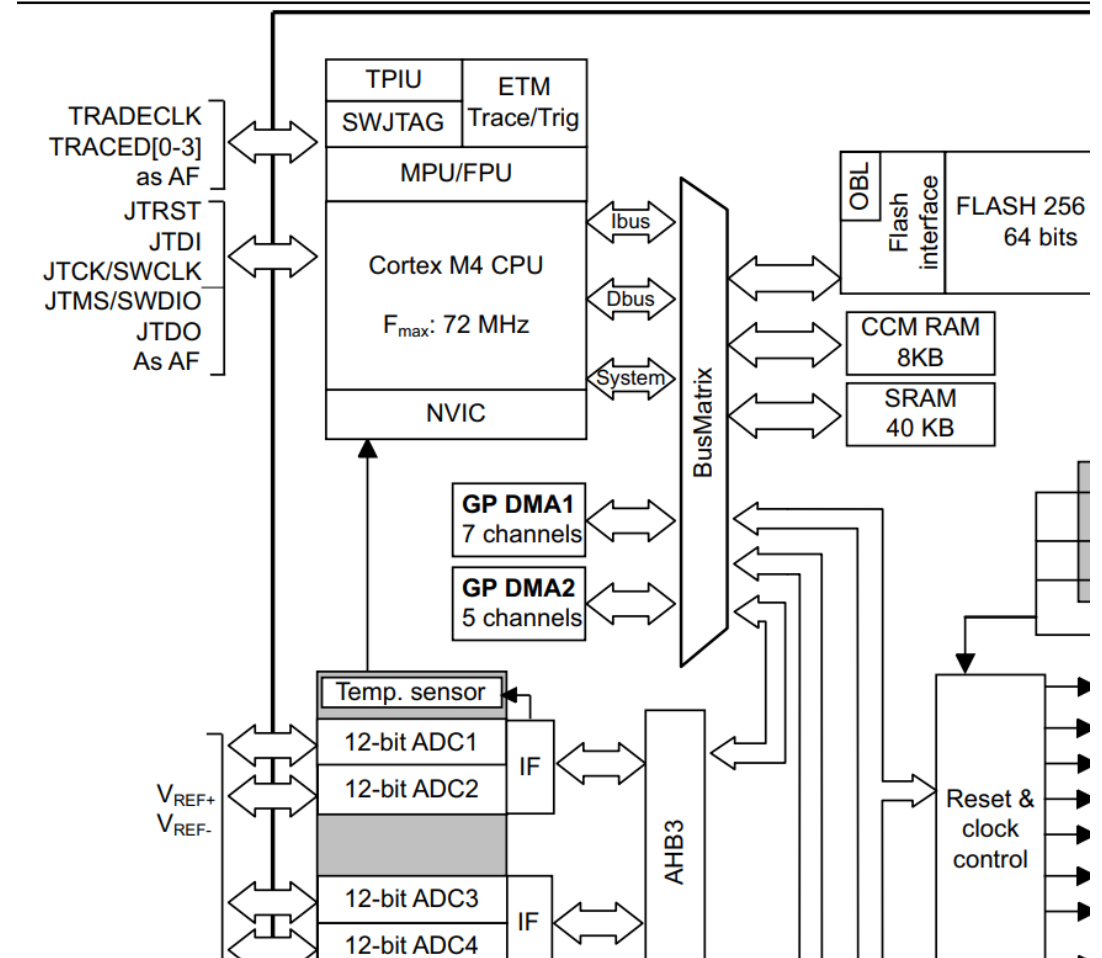
- Separate code & data busses
- Hardcoded access logic – addresses overlap
 - Address 0x0000 exists on both memories
 - Impossible to execute from SRAM
- E.g., AVR (ATMega328P at right)



Modified Harvard Architecture

- One address space
 - Possible to execute from any memory
- Separate code & data busses
 - Enables more efficient caching
- E.g., Cortex-M (STM32F3 at right)
- Most microcontrollers are this now

Figure 1. STM32F303xB/STM32F303C8



Memory Behaviour

- Different memory will have different times
- What happens if memory is not fast enough?
 - *CPU Stall*

Memory-Mapped I/O

The following table gives the boundary addresses of the peripherals available in the devices.

Table 2. STM32F303xB/C and STM32F358xC peripheral register boundary addresses⁽¹⁾

Bus	Boundary address	Size (bytes)	Peripheral	Peripheral register map
AHB3	0x5000 0400 - 0x5000 07FF	1 K	ADC3 - ADC4	Section 15.7 on page 416
	0x5000 0000 - 0x5000 03FF	1 K	ADC1 - ADC2	
-	0x4800 1800 - 0x4FFF FFFF	~132 M	Reserved	-
AHB2	0x4800 1400 - 0x4800 17FF	1 K	GPIOF	Section 11.4.12 on page 246
	0x4800 1000 - 0x4800 13FF	1 K	GPIOE	
	0x4800 0C00 - 0x4800 0FFF	1 K	GPIOD	
	0x4800 0800 - 0x4800 0BFF	1 K	GPIOC	
	0x4800 0400 - 0x4800 07FF	1 K	GPIOB	
	0x4800 0000 - 0x4800 03FF	1 K	GPIOA	
-	0x4002 4400 - 0x47FF FFFF	~128 M	Reserved	
AHB1	0x4002 4000 - 0x4002 43FF	1 K	TSC	Section 19.6.11 on page 510
	0x4002 3400 - 0x4002 3FFF	3 K	Reserved	-
	0x4002 3000 - 0x4002 33FF	1 K	CRC	Section 6.4.6 on page 96
	0x4002 2400 - 0x4002 2FFF	3 K	Reserved	-
	0x4002 2000 - 0x4002 23FF	1 K	Flash interface	Section 4.6 on page 85
	0x4002 1400 - 0x4002 1FFF	3 K	Reserved	-
	0x4002 1000 - 0x4002 13FF	1 K	RCC	Section 9.4.14 on page 168
	0x4002 0800 - 0x4002 0FFF	2 K	Reserved	-
	0x4002 0400 - 0x4002 07FF	1 K	DMA2	Section 13.6.7 on page 288
	0x4002 0000 - 0x4002 03FF	1 K	DMA1	
-	0x4001 8000 - 0x4001 FFFF	32 K	Reserved	-

Memory Alignment

13

Byte 3		Byte 2		Byte 1		Byte 0		
1	word (32) 0x20000000						0x20000000	
2	half word(16) 0x20000004		3	byte (8) 0x20000006		4	word ... 0x20000007	0x20000004
...word(32)						5	byte (8) 0x2000000b	0x20000008
6	byte (8) 0x2000000c		7	byte (8) 0x2000000d		8 half word(16) 0x2000000e		0x2000000c
								0x20000010
								0x20000014
								0x20000018

Unaligned access

ned and unaligned memory access

Memory Alignment

Introduction to STM32 MCU Portfolio

13

	Byte 3	Byte 2	Byte 1	Byte 0	
1	word (32)				0x20000000
2	half word (16)		byte (8)		0x20000004
3			byte (8)		0x20000006
4	word (32)				0x20000008
5	byte (8)	byte (8)	byte (8)		0x2000000c
6		byte (8)	byte (8)		0x2000000d
7			byte (8)		0x2000000e
8	half word (16)				0x20000010
					0x20000014
					0x20000018

Aligned access

	Byte 3	Byte 2	Byte 1	Byte 0	
1	word (32)				0x20000000
2	half word (16)		byte (8)	word ...	0x20000004
3			byte (8)		0x20000006
4				word ...	0x20000007
5				byte (8)	0x2000000b
6	byte (8)	byte (8)		half word (16)	0x2000000c
7		byte (8)			0x2000000d
8					0x2000000e
					0x20000010
					0x20000014
					0x20000018

Unaligned access

Figure 7: Difference between aligned and unaligned memory access

Misaligned Memory Access

- Some devices will throw a fault exception when accessing unaligned data, e.g.:

```
asm volatile("ldr r0, =0x20001001");  
asm volatile("ldr r1, [r0,#0]");
```

- Cortex-M0, M1 do not support unaligned access like above
- Cortex-M3 do work, but hardware is doing multiple accesses (slower)

Generic Aligned Access – Normal Read

```
C++ source #1
1 #include <stdio.h>
2
3 int main(void){
4
5     int i = *(unsigned int *)(0x20000000);
6     printf("%d", i);
7
8 }
```

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknow -Os
1 .LC0:
2     .ascii "%d\000"
3 main:
4     mov     r3, #33554432
5     push    {r4, lr}
6     ldr     r1, [r3]
7     ldr     r0, .L3
8     bl      printf
9     mov     r0, #0
10    pop     {r4, lr}
11    bx      lr
12 .L3:
13     .word   .LC0
```

Generic Unaligned Access – Half Off

```
#include <stdio.h>
```

```
int main(void){
```

```
    int i = *(unsigned int *) (0x2000002);
```

```
    printf("%d", i);
```

```
}
```

A ▾ ⚙ ▾ ▾ ▾ 🔧 + ▾ 🔧 ▾

1 .LC0:

2 .ascii "%d\000"

3 main:

4 mov r3, #33554432

5 push {r4, lr}

6 ldrh r2, [r3, #2]

7 ldrh r1, [r3, #4]

8 ldr r0, .L3

9 orr r1, r2, r1, lsl #16

10 bl printf

11 mov r0, #0

12 pop {r4, lr}

13 bx lr

14 .L3:

15 .word .LC0

Generic Aligned Access – Half Off



Generic Unaligned Access – 1 Off

C++

```
1  #include <stdio.h>
2
3  int main(void){
4
5      int i = *(unsigned int *)(0x2000001);
6      printf("%d", i);
7
8  }
```

ARM GCC 15.2.0 (unknown) -Os

```
1  .LC0:
2      .ascii "%d\000"
3  main:
4      mov     r2, #33554432
5      ldrb    r1, [r2, #2]    @ zero_extendqisi2
6      ldrb    r3, [r2, #1]    @ zero_extendqisi2
7      push    {r4, lr}
8      orr     r3, r3, r1, lsl #8
9      ldrb    r1, [r2, #3]    @ zero_extendqisi2
10     orr     r3, r3, r1, lsl #16
11     ldrb    r1, [r2, #4]    @ zero_extendqisi2
12     ldr     r0, .L3
13     orr     r1, r3, r1, lsl #24
14     bl      printf
15     mov     r0, #0
16     pop     {r4, lr}
17     bx      lr
18 .L3:
19     .word   .LC0
```

Generic Unaligned Access – 1 Off

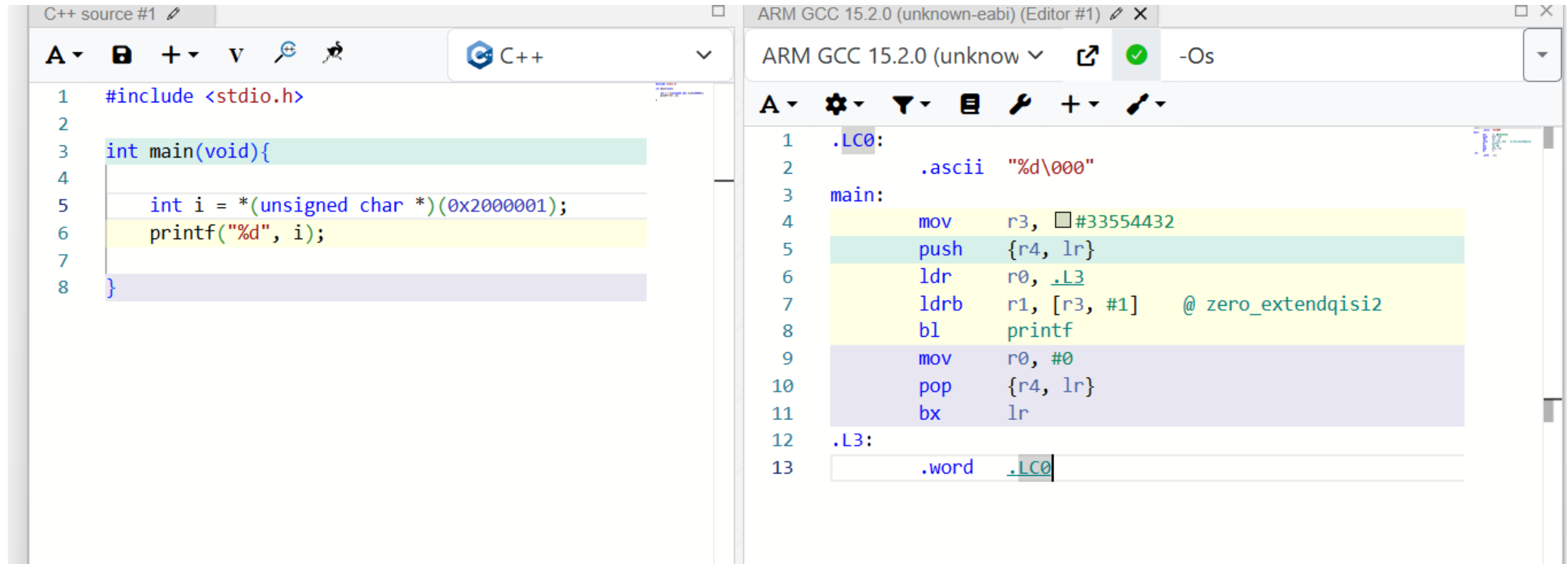
C++

```
1  #include <stdio.h>
2
3  int main(void){
4
5      int i = *(unsigned int *)(0x2000001);
6      printf("%d", i);
7
8  }
```

ARM GCC 15.2.0 (unknown) -Os

```
1  .LC0:
2      .ascii "%d\000"
3  main:
4      mov     r2, #33554432
5      ldrb    r1, [r2, #2]    @ zero_extendqisi2
6      ldrb    r3, [r2, #1]    @ zero_extendqisi2
7      push    {r4, lr}
8      orr     r3, r3, r1, lsl #8
9      ldrb    r1, [r2, #3]    @ zero_extendqisi2
10     orr     r3, r3, r1, lsl #16
11     ldrb    r1, [r2, #4]    @ zero_extendqisi2
12     ldr     r0, .L3
13     orr     r1, r3, r1, lsl #24
14     bl      printf
15     mov     r0, #0
16     pop     {r4, lr}
17     bx      lr
18 .L3:
19     .word   .LC0
```

Generic Unaligned access – 1 byte



```
C++ source #1
1  #include <stdio.h>
2
3  int main(void){
4
5      int i = *(unsigned char *) (0x2000001);
6      printf("%d", i);
7
8  }
```

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknown-eabi) -Os
1  .LC0:
2      .ascii "%d\000"
3  main:
4      mov     r3, #33554432
5      push    {r4, lr}
6      ldr     r0, .L3
7      ldrb    r1, [r3, #1]    @ zero_extendqisi2
8      bl      printf
9      mov     r0, #0
10     pop     {r4, lr}
11     bx      lr
12  .L3:
13     .word    .LC0
```

Single Byte Access Without ldrb



Memory Struct Padding / Layout

The image displays a C++ IDE with two panels. The left panel shows the source code for a C++ program, and the right panel shows the corresponding assembly code generated by ARM GCC 15.2.0 (unknown-eabi).

Source Code (Left Panel):

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 typedef struct {
5     uint8_t header;
6     uint32_t crc;
7     uint8_t length;
8 } protocol_t;
9
10 protocol_t buf;
11
12 int main(void){
13
14     buf.header = 0xA5;
15     buf.crc = 0x02;
16     buf.length = 0x11;
17
18 }
```

Assembly Code (Right Panel):

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknow -O1

1 main:
2     ldr    r3, .L2
3     mvn    r2, #90
4     strb   r2, [r3]
5     mov    r2, #2
6     str    r2, [r3, #4]
7     mov    r2, #17
8     strb   r2, [r3, #8]
9     mov    r0, #0
10    bx     lr
11 .L2:
12     .word  .ANCHOR0
13     .set   .ANCHOR0, . + 0
14 buf:
15     .space 12
```

Memory Struct Padding / Layout

```
1  #include <stdio.h>
2  #include <stdint.h>
3
4  typedef struct {
5      uint8_t header;
6      uint32_t crc;
7      uint8_t length;
8  } __attribute__((packed)) protocol_t;
9
10 protocol_t buf;
11
12 int main(void){
13
14     buf.header = 0xA5;
15     buf.crc = 0x02;
16     buf.length = 0x11;
17
18 }
```

```
ARM GCC 15.2.0 (unknown) -O1
1  main:
2      ldr    r3, .L2
3      mvn    r2, #90
4      strb   r2, [r3]
5
6      mov    r0, #0
7      mov    r2, #2
8      strb   r2, [r3, #1]
9      strb   r0, [r3, #2]
10     strb   r0, [r3, #3]
11     strb   r0, [r3, #4]
12
13     mov    r2, #17
14     strb   r2, [r3, #5]
15
16     bx     lr
17
18 .L2:
19     .word   .LANCHOR0
20     .set    .LANCHOR0, . + 0
21
22 buf:
23     .space  6
```

Struct Packing

- When dealing with protocols & structs – be very careful ensuring this is working as intended.
- Quick tip: use sizeof() on the struct to detect packing:

```
2  #include <stdint.h>
3
4  typedef struct {
5      uint8_t header;
6      uint32_t crc;
7      uint8_t length;
8  } __attribute__((packed)) protocol_t;
9
10 protocol_t buf;
11
12 int main(void){
13
14     volatile int sz = sizeof(protocol_t);
15 }
```

```
1  main:
2      sub    sp, sp, #8
3
4      mov    r3, #6
5      str    r3, [sp, #4]
6
7      ldr    r3, .L3
8      mvn    r2, #90
9      strb   r2, [r3]
10
11     mov    r0, #0
12     mov    r2, #2
13     strb   r2, [r3, #1]
14     strb   r0, [r3, #2]
15     strb   r0, [r3, #3]
16     strb   r0, [r3, #4]
17     mov    r2, #17
```

Endianness – Little Endian Byte Ordering

0x20001000 = EE **Least Significant Byte (LSB)**

0x20001001 = 5D

0x20001002 = 04

0x20001003 = F7 **Most Significant Byte (MSB)**

Address: 0x20001000 Size: auto	
20001000	F7045DEE
20001004	B75C41C8
20001008	C215C947
2000100C	507E51E8
20001010	4BF8F388
20001014	8AD17A54
20001018	A2055766
2000101C	AA735032
20001020	5757652E
20001024	D2432A56
20001028	00500000

: i

Address: 0x20001000 Size: auto	
20001000	EE 5D 04 F7
20001004	C8 41 5C B7
20001008	47 C9 15 C2
2000100C	E8 51 7E 50
20001010	88 F3 F8 4B
20001014	54 7A D1 8A
20001018	66 57 05 A2
2000101C	32 50 73 AA
20001020	2E 65 57 57
20001024	56 2A 43 D2

Network & Host Order

- In computer programs we often have a ***host order*** representing the native format that load & store will use.
- If we need to send out on a protocol (“network”), we need to convert to ***network order***.
- You may run across “host to network” and “network to host” macros that do this.

e.g., htons() macro in some random code:

```
address.sin_addr.s_addr = INADDR_ANY;  
address.sin_port = htons(PORT);
```

```
serv_addr.sin_family = AF_INET;  
serv_addr.sin_port = htons(PORT);
```

Endianness – Bit Ordering

- Be aware processors can use *big endian* – in which case watch carefully in datasheets, as for example peripheral bit maps will look “backwards”. *MPC5676R, PowerPC, example:*

immediately to the MMU.

Offset: SIU_A_BASE + 0x09B4

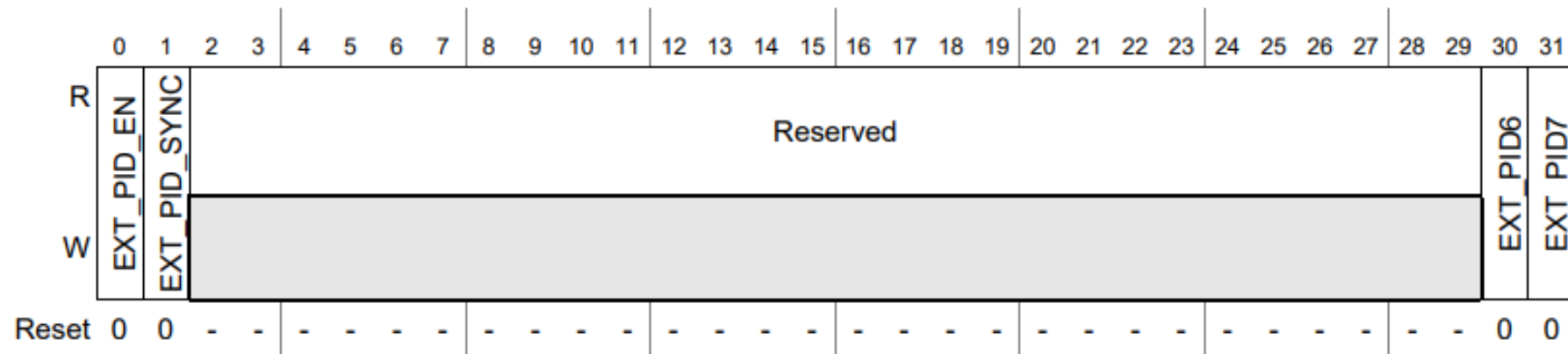


Figure 3-36. Core 0 PID Mapping Control Register (SIU_C0PID)

Memory Reordering

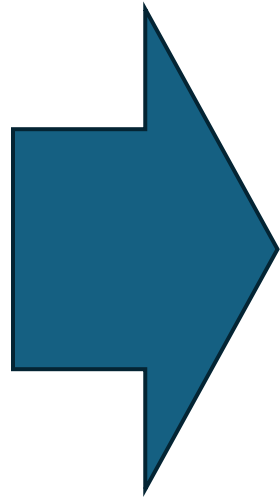
```
i = x + y;  
y = 25 + d;
```



```
ldr r0, [x]  
ldr r1, [y]  
add r0, r0, r1  
str [i], r0  
ldr r2, d  
add r2, #25  
str [y], r1
```

Memory Reordering

```
i = x + y;  
y = 25 + d;
```



```
ldr r0, [x]  
ldr r1, [y]  
add r0, r0, r1  
str [i], r0  
ldr r2, d  
add r2, #25  
str [y], r1
```

Load [x]

Load [y]

Store [i]

Load [d]

Store [y]

Memory Re-Ordering

- Provided functionality is not impacted, memory access order may be optimized.
 - Often this means accesses may be “merged”
- This may happen especially due to slow memory that takes longer than a single cycle (otherwise processor stalls).
- Arm calls this memory type “***normal memory***”.

Special Memory Types

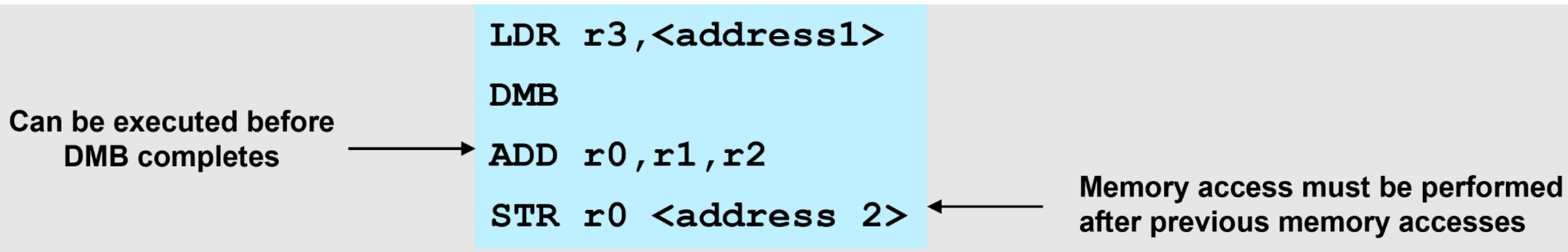
- Arm defines a type of “***device memory***”
 - Think of peripherals where you need access to occur in the right order – the actual write (or sometimes read) may have side effects or otherwise needs to happen in the program order.
 - Device memory can have different attributes allowing fine-tuning of these settings.

Barriers

- Barriers provide special instructions to block execution until conditions met.
- Data Memory Barrier
- Data Synchronization Barrier
- Instruction Synchronization Barrier

Data Memory Barrier (DMB)

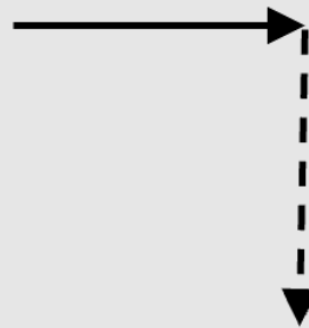
- Pauses program at the *next memory access* if the previous one isn't complete



Data Synchronization Barrier (DSB)

- Pauses program execution until memory accesses are complete.

**Cannot be executed until
DSB completes**



```
LDR r3,<address1>
```

```
DSB
```

```
ADD r0,r1,r2
```

```
STR r0 <address 2>
```

Instruction Synchronization Barrier (ISB)

- This is not related to memory – this flushes the *pipeline* which will discuss later.
- Included here to complete the types...

Summary

- Memory is a critical component of our computer.
- More than just a data store – we use memory and addressing to **interact with the world (I/O)**.
- Addresses are a **construct** – we map different blocks into addresses.
- Memory **alignment** is an important concept, one you will run into many times in your programming life.
- **Endianness** defines the byte (or bit) order